

# Task-1 DTW

## Part B

**Describe DTW:** *In your own words, explain what DTW is.*

Dynamic time warping (DTW) is an algorithm that, in essence, addresses a commonplace task in computer science: pattern matching. More specifically, DTW is a unique algorithm that allows for a more holistic analysis of two time series that may be similar in shape but misaligned temporally. The problem DTW seeks to compute is one lodged in the principles of optimality, where it ultimately seeks to find the optimal alignment of two sequences, such that their cost of alignment is minimal, that is to say, the most similar vertices between the two sequences are correctly compared, to give an analysis of the similarity of their values independent from their misalignment on the temporal axis. It achieves this through a series of minimalised “insertions”, “deletions”, and “matches” that transform the series’ being compared.

**Explain how DTW differs from simple sequence matching:** *Highlight the unique aspects of DTW that allow it to handle variations in speed and timing.*

Sequence matching is limited by the specific characteristics being compared and, in general, does so, utilising a simple approach without regard to a goal of optimality or minimisation. Commonly this may be done by comparing each corresponding  $i_{th}$  and  $j_{th}$  element in two sequences while possible and taking the Euclidean distance between each node as a way of ascertaining the similarity between two sequences. This basic approach falls short in cases of temporal misalignment. That is to say, imagine a recording of two identical heartbeats, take the case where the recording recorded is started at a different point in the heartbeat pattern, and as a result, a simple sequence match may report them as different when they are, in fact, the same. It is this conundrum that, as mentioned above, DTW attempts to mitigate. What DTW attempts to do is match each vertex with its corresponding one in the other sequence, that is to say, it aligns based on minimising the overall distance between compared nodes, eg. the  $i_{th}$  of one sequence might match the  $(i+1)_{th}$  of the other. As mentioned above, it accomplishes this through manipulation of the time axis, using a series of insertions (aligning a node in *seq1* with a corresponding gap in *seq2*), deletions (deleting a node in *seq1* to match a gap in *seq2*), and matches (directly aligned with a corresponding node temporally).

This approach gives DTW flexibility and applicability that simple sequence matching does not have.

**Detail the initial setup of the cost matrix:** *Describe what each cell represents and how the initial values are set.*

The size of the cost matrix is determined given two time series  $S^1$  and  $S^2$  of length  $n$  and  $m$ , respectively, to be a two-dimensional matrix of size  $(n+1) * (m+1)$ . Each cell within the matrix  $[i, j]$  represents the cost of aligning the  $i$ th element of  $S^1$  ( $1 \leq i < n + 1$ ), with

the corresponding  $j$ th element of  $S^2$  ( $1 \leq j < m + 1$ ). Notably, the size of the matrix is larger than the dimensions, this is to facilitate the DTW recurrence relation that will be discussed later, but for now,  $[0, 0]$  is initialised to zero to represent a start point of zero alignment cost, and the other values in the 0th row and column are initialised to infinity to ensure they are not considered in the minimisation problem. Typically in implementation but less integral to the recurrence relation the remaining values may also be initialised to infinity to indicate the current non-existence of an alignment path.

**Describe the process of calculating the optimal path through the cost matrix:** Explain the criteria used to determine the path (e.g., minimising total cost).

As mentioned, DTW is built on the idea of optimality and takes a series of optimal steps to evaluate a minimised answer cumulatively. It does this by building each optimal path from the optimal path of its predecessors, that is to say  $[i, j]$  value in a given cost matrix will be based on the minimum cost of its predecessor cells,  $[i, j - 1]$ ,  $[i - 1, j - 1]$ ,  $[i - 1, j]$  and adding it to the Euclidean distance of the  $i$ th and  $j$ th element in each sequence. This approach facilitates a dynamic programming approach where the minimal cost of alignment is to be found in the  $[n, m]$  index of the matrix. So then, retroactively using the principles that were used to build the matrix, the optimal path can also be traced by choosing the minimal cost of each cell's predecessor cells from the  $[n, m]$  index until the start point is reached.

## Part C

**Recurrence Relation:** Explain the recurrence relation used in DTW, describing how it facilitates the dynamic programming approach to find the minimal distance.

The cost matrix is formed utilising the following recurrence relation

$$cost_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ and } j = 0 \\ \infty & \text{if } i = 0 \text{ and } j > 0 \\ \infty & \text{if } i > 0 \text{ and } j = 0 \\ distance(S_i^1, S_j^2) + \min \begin{cases} cost_{i-1,j-1} \\ cost_{i,j-1} \\ cost_{i-1,j} \end{cases} & \text{if } i > 0 \text{ and } j > 0 \end{cases}$$

In general, dynamic programming is used to help solve problems with a lower time complexity than recursion when the recurrence relation can reduce a larger problem into smaller subproblems that overlap and preserve an optimal substructure. That is, an optimal solution can be constructed from optimal solutions to smaller subproblems. This plays into the idea of optimality and trading space complexity in order to have a lower time complexity. In essence, DTW is a problem of optimality where the optimal distance at a position  $i,j$  is formulated and constructed from the minimal solution of its predecessors. Looking at the recurrence relation, we see the 3 base cases of initialisation, initialising the first row and

column to infinity (the cost of aligning any value in S1 or S2 with a non-existent value in the other sequence) and 0,0 to 0 (the start representing a cost of zero). Then as  $i > 0$  &  $j > 0$ , each index is calculated, as discussed earlier, the sum of the distance between the respective elements in the sequence and the minimal alignment cost of its three predecessors. As can be now seen by implementing a bottom-up approach [1, 1] can be calculated as  $\text{distance}(S_1^1, S_2^1) + \min(0, \infty, \infty)$ , and by effect of each other subsequent values within the matrix can be calculated. So by solving each minimal cost, we facilitate the solving of the larger subproblem.

## Part E

**Computational complexity:** Discuss the computational complexity of the algorithm implemented in Part D (i.e., DTW with boundary constraints). Is it different from the algorithm in Part A (i.e., standard DTW)? State and explain the recurrence relation.

If we first consider the computation complexity of DTW, the space complexity of DTW will given by the space used by the matrix + some trivial constant, considering that the same matrix is formed each time, space complexity can be described by the space complexity,  $\theta((n + 1) * (m + 1)) \approx \theta(n * m)$

Furthermore, if we take the basic operation to be the updating of the matrix with a minimum cost as a constant operation, it can be said that for the given matrix, the

*number of updated values is  $(n + 1) * (m + 1) \Rightarrow \theta((n + 1) * (m + 1)) \approx \theta(n * m)$*

Now comparatively to the constriction suggested in part D how will this look. Consider that between the two parts, the only dissimilarity will be the restriction of the window size parameter win. Consider the following recurrence relation that assumes  $n = m$ , that is to say, the matrix is square and has a discernible diagonal,

Assuming  $i \in [j - \text{win}, j + \text{win}]$

$$\text{cost}_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ and } j = 0 \\ \infty & \text{if } i = 0 \text{ and } j > 0 \\ \infty & \text{if } i > 0 \text{ and } j = 0 \\ \text{distance}(S_i^1, S_j^2) + \min \begin{cases} \text{cost}_{i-1,j-1} \\ \text{cost}_{i,j-1} \\ \text{cost}_{i-1,j} \end{cases} & \text{if } i, j > 0 \end{cases}$$

That is, the values are only updated within the matrix given that they fall within the window size highlighted above. Using this, the computational complexity can be analysed, for a normal implementation, the space complexity remains the same,  $\theta((n + 1) * (m + 1)) \approx \theta(n * m)$

Conversely if the time complexity is to be considered, taking the same basic operation as highlighted above, the matrix will be updated for all values that fall within the window that is they fall within a portion of size  $2\text{win} + 1$  of each matrix row, for  $n + 1$  columns, so

Assuming a square matrix of size  $n+1$ , and where  $win$  = window size

$$\begin{aligned}
 \text{number of updated values is } & (n+1) * (\min(2 * win + 1, n+1)) - win(win+1) \\
 \Rightarrow & O((n+1) * (\min(2 * win + 1, n+1))) \\
 \approx & O(n * \min(win, n)), \text{ Note as win} \\
 \rightarrow & \frac{n}{2}, \text{ complexity returns to normal DTW complexity}
 \end{aligned}$$

when  $win = 0$  (minumum value), time complexity is  $\Omega(n+1)$ , elems on diagonal, subbing that into above  $O$  bound yields

$$\begin{aligned}
 \text{number of updated values is } & (n+1) * (\min(2 * win + 1, n+1)) - win(win+1) \\
 \Rightarrow & O((n+1) * (\min(2 * (0) + 1, n+1)) - (0)((0)+1)) \approx O(n+1)
 \end{aligned}$$

Upper bound = lower bound  $\Rightarrow \Theta(O(n * \min(win, n)))$

## Part G

The computation complexity of the problem addressed in part f, is somewhat harder to address and is heavily implementation-dependent, as such, while and upper bound can be provided, quantifying a lower bound can be trickier as it is heavily dependent on how optimised the algorithm is, in general, an upper bound space complexity for implementation via a 3d array can be given as such, where  $k$  reflects the allowed path length,

$$O((n+1) * (m+1) * k) \approx O(n * m * k)$$

While in a basic interpretation, this could also reflect the theta bound if the 3d matrix was scaled in accordance to the allowed length a smaller theta bound could perhaps be achieved. Furthermore, the same logic follows for time complexity, in general, it can be also upper bounded considering the space complexity,

$$\begin{aligned}
 \text{number of updated values is } & \leq (n+1) * (m+1) * (k) \Rightarrow O((n+1) * (m+1) * k) \\
 & \approx O(n * m * k)
 \end{aligned}$$

In reality, depending on the implementation and basic operation taken, this could potentially be made stricter. Similarly, the recurrence relation is ultimately subject to the implementation but, in this case, can be given by,

$$cost_{i,j,k} = \begin{cases} 0 & \text{if } i,j,k = 0 \\ \infty & \text{if } i = 0 \text{ and } j > 0 \\ \infty & \text{if } i > 0 \text{ and } j = 0 \\ \infty & \text{if } i,j > k \quad i,j \leq k \\ distance(S_i^1, S_j^2) + \min \begin{cases} cost_{i-1,j-1,k-1} \\ cost_{i,j-1,k-1} \\ cost_{i-1,j,k-1} \end{cases} & \text{if } i,j,k > 0 \quad i,j \leq k \end{cases}$$

Where each minimum cost at i,j,k is constructed from the minimum cost in the predecessor cells of a previous path length. And set as infinity in all places not considered in the current path length. This is also where one improvement could be made in regard to the matrix's being constructed dependent on the len k for all while  $k \leq i,j$ .